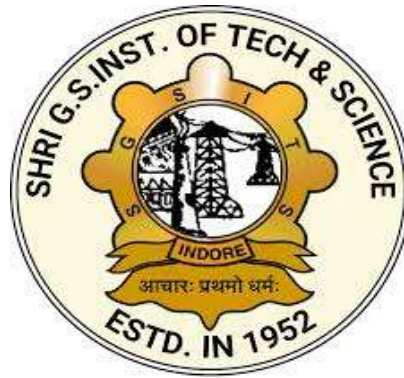


**SHRI G.S. INSTITUTE OF TECHNOLOGY & SCIENCE,
INDORE**



DEPARTMENT OF INFORMATION TECHNOLOGY

Minor Project Report

On

mlnpm: A Machine Learning Package Manager

Supervised by:

Mr. Manjeet K. Soni

Associate Professor

Department of Information Technology

Co-Supervised by:

Ms. Prapti Godeshwar

Assistant Professor

Department of Information Technology

Submitted by:

Bhavika Lohia

0801IT231041

Information Technology

Kanishka Sharma

0801IT231066

Information Technology

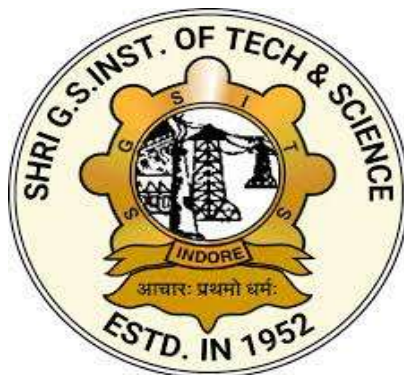
Neel Desai

0801IT231085

Information Technology

**SHRI G.S. INSTITUTE OF TECHNOLOGY & SCIENCE,
INDORE**

DEPARTMENT OF INFORMATION TECHNOLOGY



RECOMMENDATION

We are pleased to recommend that the Minor Project Report entitled "mlnpm: A Machine Learning Package Manager" submitted by Bhavika Lohia, Kanishka Sharma, and Neel Desai be accepted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal (M.P.), during the academic session 2026.

Mr. Manjeet K. Soni
Associate Professor
Information Technology

Prof. K. K. Sharma
Head of Department
Professor
Information Technology

**SHRI G.S. INSTITUTE OF TECHNOLOGY & SCIENCE,
INDORE**

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the Minor Project entitled "mlnpm: A Machine Learning Package Manager" submitted by Bhavika Lohia, Kanishka Sharma, and Neel Desai to the Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal, during the academic session 2026-2027 is a genuine work performed by them.

Internal Examiner

Date:

External Examiner

Date:

DECLARATION

We, Bhavika Lohia, Kanishka Sharma, and Neel Desai, undergraduate students in the Department of Information Technology, S.G.S.I.T.S., Indore (M.P.), hereby declare that the Minor Project titled "mlnpm: A Machine Learning Package Manager" is our original work, carried out under the supervision of **Mr. Manjeet K. Soni, Associate Professor, Department of Information Technology, S.G.S.I.T.S., Indore (M.P.).**

We further declare that, to the best of our knowledge, this project report does not contain any material previously submitted for the award of any degree or diploma in this or any other university. Any sources of information used have been properly acknowledged and cited.

Signature of the candidate:

Name of the candidate: Bhavika Lohia

Enrolment No.: 0801IT231041

Signature of the candidate:

Name of the candidate: Kanishka Sharma

Enrolment No.: 0801IT231066

Signature of the candidate:

Name of the candidate: Neel Desai

Enrolment No.: 0801IT231085

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide, **Dr. Upendra Singh, Assistant Professor**, for his valuable guidance, continuous support, and encouragement throughout the completion of this project.

We are also thankful to the **Head of the Department, Prof. K. K. Sharma**, for providing the necessary facilities and support to carry out this work.

We extend our heartfelt thanks to the **Dean (ARSD), Prof. Lalit Purohit**, for his encouragement and institutional support.

Finally, we are grateful to the faculty members and technical staff of the **Department of Information Technology, S.G.S.I.T.S., Indore**, for their support and cooperation.

Signature:

Bhavika Lohia

0801IT231041

Signature:

Kanishka Sharma

0801IT231066

Signature:

Neel Desai

0801IT231085

ABSTRACT

The integration of machine learning models into production-grade Node.js applications has historically required complex Python microservices, heavyweight infrastructure, and significant inter-language overhead. This project presents mlnpm — a Machine Learning Package Manager — a distributed, full-stack ecosystem that enables developers to package, publish, discover, and consume machine learning models as native NPM-compatible modules with zero Python dependencies at runtime.

mlnpm comprises three tightly integrated components: a centralized global registry built on Express.js and PostgreSQL with Cloudflare R2 binary storage; a command-line interface (@mlnpm/cli) for model acquisition, caching, and CI/CD restoration; and a native runtime engine (@mlnpm/runtime) powered by ONNX Runtime Node.js bindings for hardware-accelerated inference. The system uses the ONNX format as the universal model interchange standard and supports GPU acceleration through NVIDIA CUDA, Apple CoreML, and DirectML.

The project is fully deployed at mlnpm.vercel.app, with CLI and runtime packages published to the official NPM registry. All 20 defined test cases achieved a 100% pass rate.

Keywords: Machine Learning, NPM, ONNX Runtime, Node.js, Model Registry, Package Manager, JavaScript, CLI

TABLE OF CONTENTS

Contents	Page No.
Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
List of Figures	viii
List of Tables	ix
Chapter 1: Introduction	11
1.1 Overview	11
1.2 Problem Statement	11
1.3 Objectives	12
1.4 Scope	12
Chapter 2: Literature Review	13
2.1 Package Management Systems	13
2.2 Existing ML Model Serving Solutions	13
2.3 ONNX as Universal ML Interchange Standard	14
2.4 Cloud Object Storage and Secure Binary Distribution	14
2.5 Semantic Versioning	15
Chapter 3: System Analysis	16
3.1 Existing System	16
3.2 Proposed System	16
3.3 Requirement Analysis	16
3.3.1 Functional Requirements	16
3.3.2 Non-Functional Requirements	17
3.3.3 Technology Stack Summary	18
Chapter 4: System Design	19
4.1 Architecture (Client-Server)	19
4.1.1 CLI/Runtime Internal Handshake	20
4.2 Database Design	21

4.3 ER Diagram	22
4.4 Data Flow Diagram	23
4.4.1 Level 0 – Context Diagram	23
4.4.2 Level 1 – Detailed DFD	24
Chapter 5: Implementation	25
5.1 Frontend	25
5.2 Backend	25
5.3 CLI & Runtime	25
5.3.1 @mlnpm/cli – Command Reference	25
5.3.2 Global Model Mirroring	26
5.3.3 @mlnpm/runtime – Wrapper API (Courier)	26
5.3.4 Security Implementation	27
5.4 Screenshots	28
Chapter 6: Testing	31
6.1 Test Cases	31
6.2 Results	32
Chapter 7: Conclusion & Future Scope	33
7.1 Conclusion	33
7.2 Future Scope	33
References	35
Appendix	36

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1	MLNPM System Architecture Diagram	19
Figure 2	CLI/Runtime Internal Handshake Flow	20
Figure 3	Package Installation Workflow	21
Figure 4	ER Diagram – Database Entity Relationships	22
Figure 5	Data Flow Diagram – Level 0 (Context Diagram)	23
Figure 6	Data Flow Diagram – Level 1 (Detailed)	24
Figure 7	MLNPM Web Dashboard – Home Page	28
Figure 8	Model Explore & Discovery Interface	28
Figure 9	Model Publish / Upload Interface	29
Figure 10	CLI Terminal Output – mlnpm install	29

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Comparison of mlnpm with Existing Solutions	14
Table 2	Functional Requirements	16
Table 3	Non-Functional Requirements	17
Table 4	Technology Stack Summary	18
Table 5	CLI Commands Reference	25
Table 6	Supported ONNX Tensor Data Types	26
Table 7	Security Features and Implementation	27
Table 8	Test Cases – Registry, Authentication & CLI	31

CHAPTER 1: INTRODUCTION

1.1 Overview

The rapid proliferation of machine learning in modern software has created a significant ecosystem divide. Data scientists primarily work in Python, using frameworks such as PyTorch [11], TensorFlow [10], and scikit-learn [12] to build and train models. Web developers, on the other hand, work in JavaScript and Node.js [13] for scalable backend applications. Bridging these two worlds has historically been a complex, resource-intensive undertaking.

mlnpm — the Machine Learning Package Manager — is a full-stack distributed ecosystem purpose-built to resolve this divide [15]. It enables data scientists to package and publish machine learning models as standard Node.js modules, and empowers web developers to install, manage, and consume those models as native NPM dependencies [3] — with zero Python code required at runtime. The system uses the Open Neural Network Exchange (ONNX) format [1][8] as the universal model interchange standard and is publicly accessible at mlnpm.vercel.app [7].

1.2 Problem Statement

Integrating machine learning into production Node.js applications presents several deeply intertwined challenges:

- **Cross-Language Dependency:** Deploying an ML model in Node.js traditionally requires a separate Python runtime, pip/conda package management, and a dedicated model-serving microservice (FastAPI/Flask), introducing significant operational overhead.
- **Infrastructure Overhead:** Python microservices require always-on servers, incurring hosting costs even during low-traffic periods and complicating CI/CD pipelines that must coordinate multiple language runtimes.
- **No Unified Distribution Standard:** There is no NPM-equivalent for ML models targeting the JavaScript/Node.js ecosystem. Platforms like Hugging Face do not integrate with the NPM dependency management paradigm.
- **Manual Integration Effort:** Web developers must manually handle model downloads, tensor structures, Python-to-JavaScript bridging, memory loading, and hardware acceleration — requiring deep ML expertise most web developers lack.

1.3 Objectives

1. Develop a centralized, semantically-versioned model registry with secure binary distribution via Cloudflare R2 [5].
2. Implement a CLI tool (@mlnpm/cli) providing NPM-like model lifecycle management including install, restore, uninstall, list, and cache commands [3].
3. Build a native Node.js runtime engine (@mlnpm/runtime) enabling ML inference without any Python dependency, powered by ONNX Runtime [2].
4. Define the Courier wrapper specification — a standardized interface bridging ML tensor operations and JavaScript object-oriented programming.
5. Support hardware-accelerated inference through automatic detection of NVIDIA CUDA, Apple CoreML, and DirectML providers.
6. Implement production-grade security: SHA-256 binary integrity verification, bcrypt password hashing, presigned URL streaming, and OTP email verification via Brevo [6].

1.4 Scope

The project encompasses the complete design, development, and deployment of the mlnpm ecosystem. In scope are the registry API, web dashboard, CLI, runtime engine, ONNX model support (PyTorch, TensorFlow, scikit-learn exports), both predict and stream inference modes, semantic versioning enforcement, and deployment on Vercel [7] and Render. Out of scope are model training pipelines, non-ONNX formats without prior export, browser-side inference (mlnpm is exclusively a backend technology), and private/enterprise registry features.

CHAPTER 2: LITERATURE REVIEW

2.1 Package Management Systems

The NPM (Node Package Manager) ecosystem established the foundational paradigm for declarative dependency management in JavaScript: semantic versioning (SemVer) [4], a centralized public registry, lock files for reproducibility, and postinstall lifecycle scripts [3]. mlnpm directly adopts these principles. The core challenge is that NPM is optimized for code modules, not for large binary artifacts such as ML model weights that may reach several gigabytes. PyPI (the Python Package Index) similarly provides centralized Python package distribution but cannot be consumed natively in Node.js environments. pnpm and Yarn introduced disk-efficient global content-addressable caches — mlnpm adopts a similar strategy with its `~/.mlnpm/cache` directory, downloading model binaries once per machine and scaffolding project references from the cache.

2.2 Existing ML Model Serving Solutions

Several platforms address ML model serving, each with distinct limitations relative to mlnpm's goals. Hugging Face Hub [15] hosts over 500,000 models and provides excellent Python-first APIs through the transformers library. While ONNX model hosting is supported via the optimum library, the primary Node.js consumption path is Transformers.js (@xenova/transformers) which runs models through WebAssembly — a fundamentally different execution environment that cannot access GPU hardware accelerators and is unsuitable for high-throughput server inference. TensorFlow.js [10] provides JavaScript TensorFlow execution but is tightly coupled to TensorFlow's model format and provides no model distribution infrastructure. BentoML [9] and MLflow are production-grade ML serving frameworks that offer model versioning and API generation but require Python servers and add significant infrastructure overhead — neither supports Node.js-native consumption. ONNX Runtime [2] provides the inference backend used by mlnpm. Developed by Microsoft, it offers high-performance execution across CPU, GPU, and specialized hardware via its Node.js binding (onnxruntime-node). However, it addresses only execution, not distribution, versioning, or developer experience — all of which mlnpm provides on top.

Table 1: Comparison of mlnpm with Existing Solutions

Feature	Hugging Face	TensorFlow.js	BentoML	mlnpm
Runtime Environment	Python/PyTorch	Node.js / WASM	Python Server	Native Node.js
Zero Python at Runtime	No	No	No	Yes
NPM Package Management	No	No	No	Yes
Multi-Framework Support	Yes	TF only	Yes	Yes (via ONNX)
GPU Acceleration	Yes	Limited	Yes	Yes (CUDA/CoreML)
Semantic Versioning	Yes	No	Yes	Yes
CI/CD Auto-Restore	No	No	No	Yes
Web Developer Friendly	Moderate	Moderate	Low	High

Table 1 compares mlnpm with the three primary existing solutions across eight criteria. mlnpm is the only platform that simultaneously satisfies zero Python runtime dependency, NPM-paradigm package management, multi-framework ONNX support, GPU acceleration, semantic versioning, and CI/CD auto-restoration — making it uniquely suited for the Node.js production backend use case.

2.3 ONNX as Universal ML Interchange Standard

The Open Neural Network Exchange (ONNX) format [1][8], developed by Facebook AI Research and Microsoft in 2017, provides a vendor-neutral, language-agnostic representation of machine learning models using Protocol Buffers. It supports operations from all major ML frameworks — PyTorch [11], TensorFlow [10], and scikit-learn [12] — enabling model portability across runtimes. ONNX's adoption as mlnpm's required model format ensures that models trained in any supported framework can be executed through the same runtime API, eliminating the need for multiple backends and providing a consistent developer experience.

2.4 Cloud Object Storage and Secure Binary Distribution

Distributing large ML model binaries requires infrastructure beyond traditional CDNs. Cloudflare R2 [5], S3-compatible object storage, provides presigned URL mechanisms that enable temporary authenticated access to protected objects without permanent client credentials. mlnpm uses R2 presigned URLs with 60-minute expiration windows for secure

model streaming, ensuring binaries are never directly exposed through the application server. SHA-256 integrity verification complements this by detecting any binary corruption during transit. For transactional email delivery, mlnpm uses the Brevo V3 REST API [6], which bypasses SMTP port-blocking common in cloud hosting environments.

2.5 Semantic Versioning

Semantic Versioning (SemVer) [4] defines version numbering as MAJOR.MINOR.PATCH, encoding backward compatibility guarantees. mlnpm strictly enforces SemVer with version immutability — once published, a version cannot be overwritten. This guarantees that applications depending on a specific model version receive the exact same binary on every install, critical for production reliability. Publishers can mark faulty versions as "yanked" to discourage use without deleting the immutable record.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Existing System

Prior to mlnpm, integrating ML inference into Node.js typically followed one of three patterns. The most common involves running a dedicated Python web server (FastAPI or Flask) alongside Node.js, requiring two separate runtimes, always-on infrastructure, and complex CI/CD coordination. The `@xenova/transformers` library enables Transformer models in Node.js via WebAssembly, but cannot access GPU accelerators in server environments and covers only Transformer-class models. TensorFlow.js (`@tensorflow/tfjs-node`) provides native TensorFlow execution in Node.js but is limited to TensorFlow's format and provides no distribution or package management infrastructure. All three approaches share a fundamental limitation: the absence of a unified, NPM-compatible model distribution layer — requiring developers to manually handle acquisition, storage, versioning, and loading.

3.2 Proposed System

mlnpm proposes treating machine learning models as first-class NPM-compatible packages through three integrated components. The Global Model Registry is a centralized REST API (Express.js + PostgreSQL + Cloudflare R2) managing user accounts, package metadata, versioning, binary uploads, and presigned URL generation, with a React.js web dashboard for discovery, publishing, and documentation. The `@mlnpm/cli` is a globally installed NPM package providing NPM-like commands (install, restore, uninstall, list, cache) that handle model downloading, caching, local project scaffolding, and CI/CD integration transparently. The `@mlnpm/runtime` is a server-side NPM package that loads ONNX models into memory and exposes clean JavaScript APIs (`init()`, `predict()`, `stream()`), wrapping `onnxruntime-node` with automatic hardware provider detection and graceful fallback.

3.3 Requirement Analysis

3.3.1 Functional Requirements

Table 2: Functional Requirements

ID	Requirement	Description
FR01	User Authentication	Registration with OTP email verification (10-min TTL via Brevo), login with bcrypt validation, JWT issuance.
FR02	Package Creation	Authenticated users create unique package namespaces with name and description.
FR03	Version Publishing	Publishers upload .onnx + wrapper.config.js files for new SemVer versions; immutability enforced.

FR04	Version Yanking	Package maintainers mark specific versions as yanked to signal consumers to avoid them.
FR05	Model Discovery	All visitors browse, search, and filter published packages by name/description/popularity.
FR06	Model Installation	CLI downloads model files, verifies SHA-256, caches globally, and scaffolds project node_modules.
FR07	Model Restoration	CLI restores all declared model dependencies in fresh environments (CI/CD pipelines).
FR08	Runtime Inference	Runtime exposes predict() and stream() methods accepting JavaScript objects, returning inference results.
FR09	Hardware Acceleration	Runtime auto-detects CUDA, CoreML, or DirectML; falls back to CPU gracefully.
FR10	Cache Management	CLI provides commands to list and clean the global model cache (~/.mlnpm/cache).

Table 2 lists the ten functional requirements of the mlnpm system. These requirements cover the complete lifecycle from user authentication and package publishing (FR01–FR04) through model discovery and installation (FR05–FR06) to runtime inference and cache management (FR07–FR10), forming the core behavioral specification of the system.

3.3.2 Non-Functional Requirements

Table 3: Non-Functional Requirements

Category	Requirement
Performance	Presigned R2 URLs for direct binary streaming (bypassing application server). API metadata responses under 500ms.
Scalability	Stateless backend (JWT + presigned URLs) enables horizontal scaling without session synchronization. B-tree indexed DB queries for search and pagination.
Security	bcrypt (12 rounds) for passwords; SHA-256 for binary integrity; Helmet.js HTTP headers; CORS origin whitelist; 10-min OTP TTL.
Reliability	Version immutability enforced at both application and DB constraint levels. Idempotent CLI operations (re-running install/restore yields consistent state).
Compatibility	Supports Node.js ≥ 16 on Windows (x64), macOS (Intel & Apple Silicon), and Linux (x64, ARM).
Maintainability	TypeScript (NodeNext module resolution) across all components. Modular environment configuration for dev/production endpoint switching.

Table 3 defines six non-functional requirements governing the quality attributes of the mlnpm system. Security, reliability, and scalability are identified as the highest-priority categories, directly influencing architectural decisions such as the stateless backend design, the presigned URL distribution model, and the SHA-256 integrity verification mechanism.

3.3.3 Technology Stack Summary

Table 4: Technology Stack Summary

Category	Technology	Purpose
Inference Engine	ONNX Runtime Node	Native OS bindings for CPU/GPU model execution
Frontend	React 18 + Vite + Tailwind CSS	Web dashboard SPA with edge deployment on Vercel
Client-Side Compiler	Sucrase	In-browser TypeScript compilation for wrapper uploads
Backend Framework	Express.js (Node.js)	REST API with JWT authentication
Database	PostgreSQL	Relational storage for users, packages, versions
Object Storage	Cloudflare R2 (S3-compatible)	Secure model binary storage and presigned streaming
Email Service	Brevo Transactional API V3	OTP delivery and notifications
HTTP Security	Helmet.js + CORS Whitelist	XSS, frame injection, CSRF protection
Package Registry	Official NPM Registry	CLI and runtime package distribution
Language	TypeScript (NodeNext)	Type-safe codebase across all components

Table 4 presents the complete technology stack adopted by mlnpm. The selection prioritizes JavaScript/TypeScript across all components (frontend, backend, CLI, runtime) to maintain a unified development experience. External services (Cloudflare R2, Brevo) are chosen for their REST-based APIs and superior reliability in cloud-hosted environments.

CHAPTER 4: SYSTEM DESIGN

4.1 Architecture (Client-Server)

mlnpm employs a distributed three-tier architecture comprising a presentation tier (React.js web dashboard), an application tier (Express.js REST API), and a data tier (PostgreSQL + Cloudflare R2). The CLI component interacts with the application tier from developer workstations and CI/CD pipelines. Figure 1 illustrates the complete system architecture.

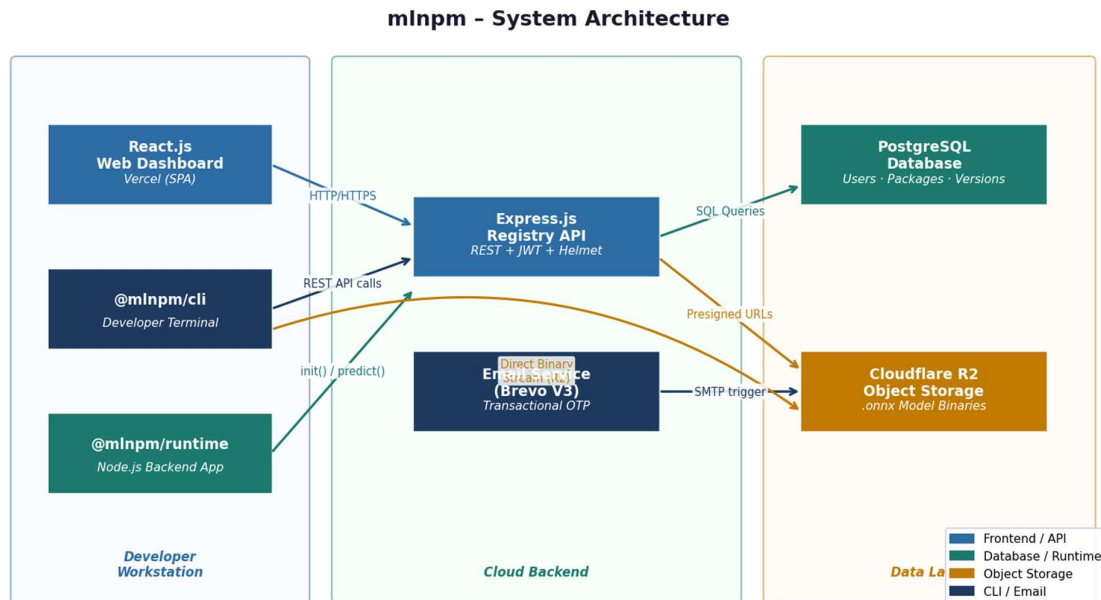


Figure 1: MLNPM System Architecture Diagram

Figure 1 illustrates the distributed three-tier architecture of mlnpm. The left zone represents developer workstations hosting the React.js Dashboard, the @mlnpm/cli, and the application's Node.js backend (where @mlnpm/runtime executes). The centre zone contains the Express.js Registry API that orchestrates all business logic. The right zone depicts the data layer comprising PostgreSQL for relational metadata and Cloudflare R2 for model binary storage. The dashed arrow from the CLI directly to R2 highlights that large model binaries are streamed directly between the developer and cloud storage, bypassing the application server to reduce latency and infrastructure load.

4.1.1 CLI/Runtime Internal Handshake

When a developer runs `mlnpm install <package>`, the CLI and runtime perform a precise sequence of operations to produce an importable Node.js module. Figure 2 details this handshake flow.

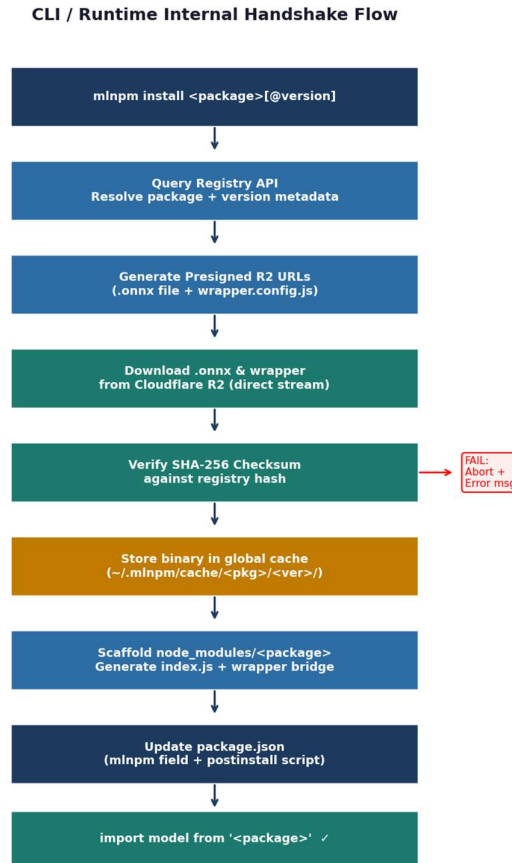


Figure 2: CLI/Runtime Internal Handshake Flow

Figure 2 depicts the sequential steps executed during `mlnpm install`. The flow begins with a registry API query to resolve package metadata, proceeds through presigned R2 URL generation and direct binary download, performs SHA-256 integrity verification (with an explicit FAIL branch that aborts installation), stores the verified binary in the global cache, dynamically scaffolds the `node_modules` entry, and concludes by updating `package.json` with the model dependency and postinstall script. This design ensures idempotency — re-running the command at any stage produces a consistent, verified final state.

The forthcoming figure shows the complete package installation workflow as a swimlane diagram, showing the interactions between all four actors — the Publisher, the mlnpm Registry API, Cloudflare R2, the Developer CLI, the Global Cache, and the project node_modules.

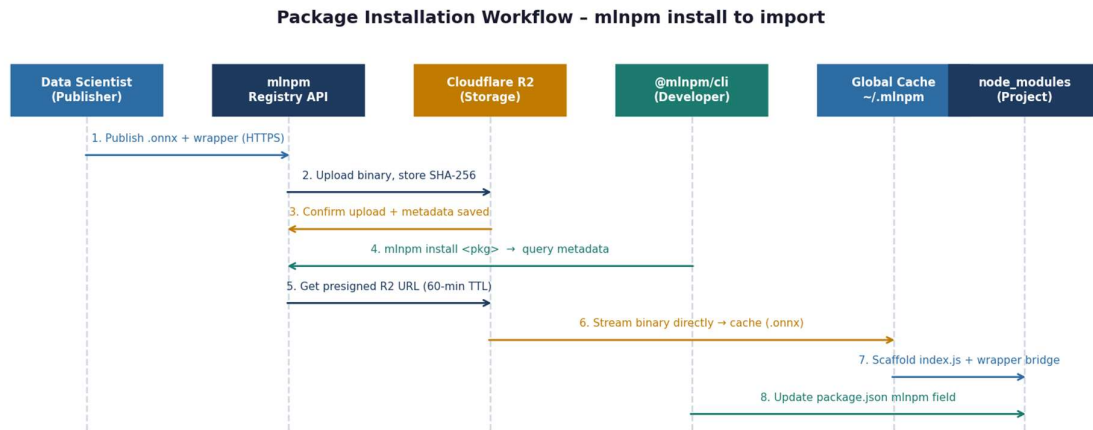


Figure 3: Package Installation Workflow – Complete Sequence from mlnpm install to importable module

Figure 3 shows the full lifecycle of a model package from publication to consumption using a swimlane diagram. Steps 1–3 represent the publisher's workflow: uploading .onnx and wrapper files to the registry, which stores binaries in R2 and records SHA-256 checksums. Steps 4–8 represent the developer's workflow: the CLI queries the registry, receives a presigned R2 URL, streams the binary directly to the global cache, scaffolds a module bridge in node_modules, and updates package.json — enabling immediate use of standard import syntax.

4.2 Database Design

The registry's data layer uses a normalized PostgreSQL schema with five primary entities. Users stores account credentials (bcrypt-hashed passwords), email verification status, and OTP data with TTL enforcement. Packages stores package metadata including name (UNIQUE, immutable), description, and owner reference with ON DELETE CASCADE from Users. Versions stores individual version records per package including the SemVer string, yanked status, and arbitrary JSON metadata (GPU requirements, citations). Version_Files stores per-file records (file name, size, SHA-256 checksum, R2 key) linked to Versions with ON DELETE CASCADE. Access_Counts tracks per-package view and install counts with B-tree indexing for O(1) popularity sorting. PL/pgSQL trigger functions automate timestamp management on every UPDATE, and OTP expiry is enforced by validating the otp_expires_at timestamp server-side on every verification request.

4.3 ER Diagram

Figure 4 depicts the complete Entity-Relationship diagram for the mlnpm registry database, showing all five entities and their relationships.

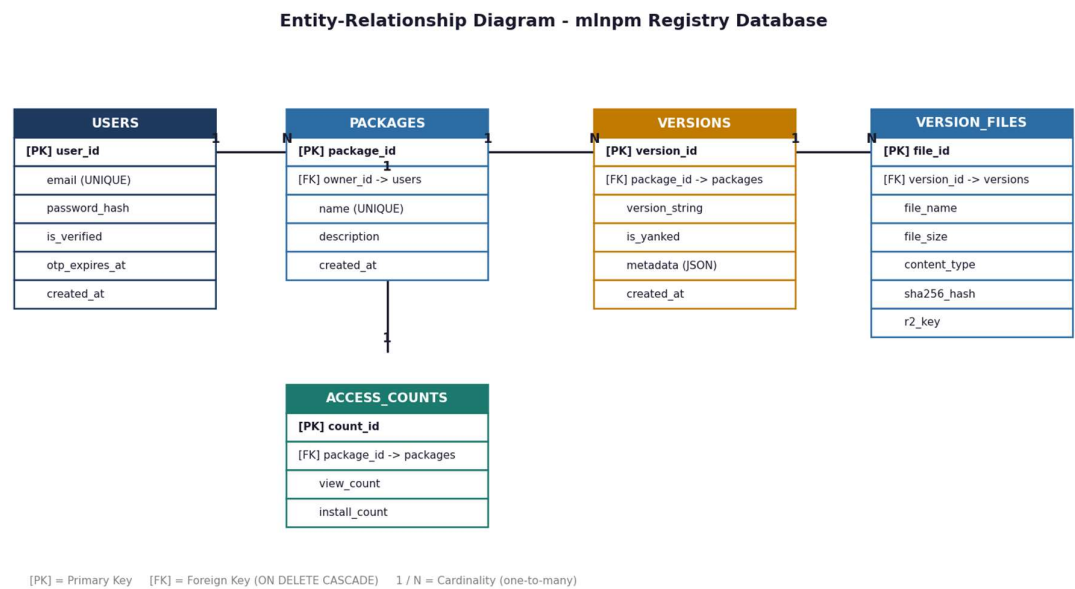


Figure 4: ER Diagram – Database Entity Relationships

Figure 4 presents the Entity-Relationship diagram of the mlnpm PostgreSQL database. The diagram reveals a hierarchical ownership structure: each User (left) may own multiple Packages (one-to-many); each Package may contain multiple Versions (one-to-many); each Version may contain multiple Version_Files (one-to-many). The Access_Counts entity maintains a one-to-one relationship with Packages for popularity tracking. All [FK] foreign key relationships are configured with ON DELETE CASCADE semantics, ensuring deep cleanup when a parent entity is deleted. [PK] denotes primary key fields, which are system-generated identifiers.

4.4 Data Flow Diagram (DFD)

4.4.1 Level 0 – Context Diagram

The Level 0 DFD treats mlnpm as a single system process and identifies its three external entities and their data flows at the highest abstraction level, as shown in Figure 5.

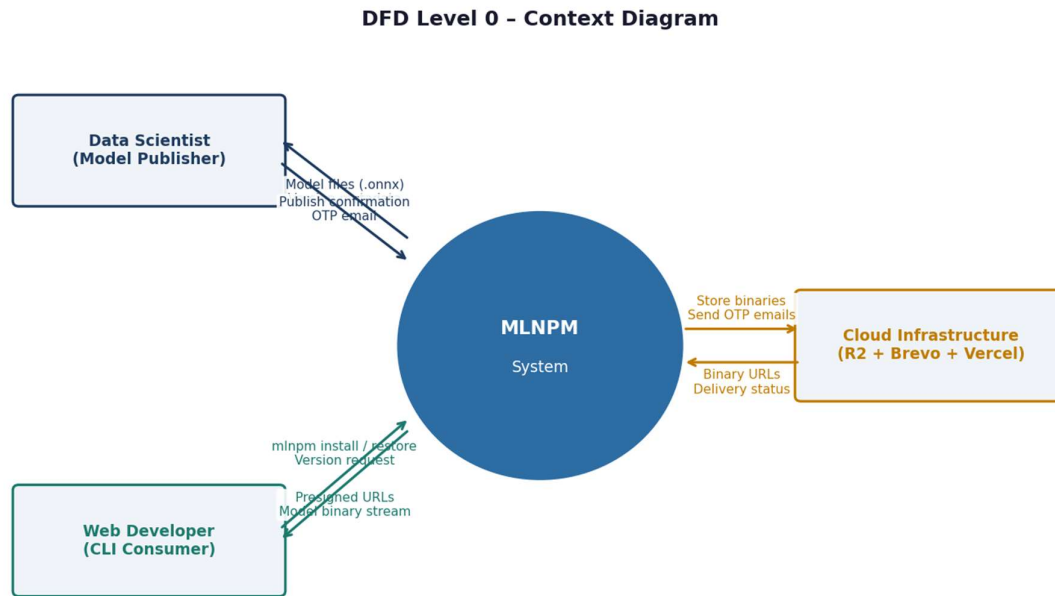


Figure 5: Data Flow Diagram – Level 0 (Context Diagram)

Figure 5 presents the Level 0 Context DFD defining mlnpm's system boundary. Three external entities interact with the system: the Data Scientist (Publisher) submits model files and version metadata and receives publish confirmations and OTP verification emails; the Web Developer (CLI Consumer) sends install/restore requests and receives presigned download URLs and model binary streams; and the Cloud Infrastructure (Cloudflare R2 + Brevo) receives binary storage instructions and email triggers, returning storage URLs and delivery confirmations. This diagram establishes the external interface contract without revealing internal system decomposition.

4.4.2 Level 1 – Detailed DFD

Figure 6 decomposes the mlnpm system into its four major functional processes and their data flows to and from the four data stores.

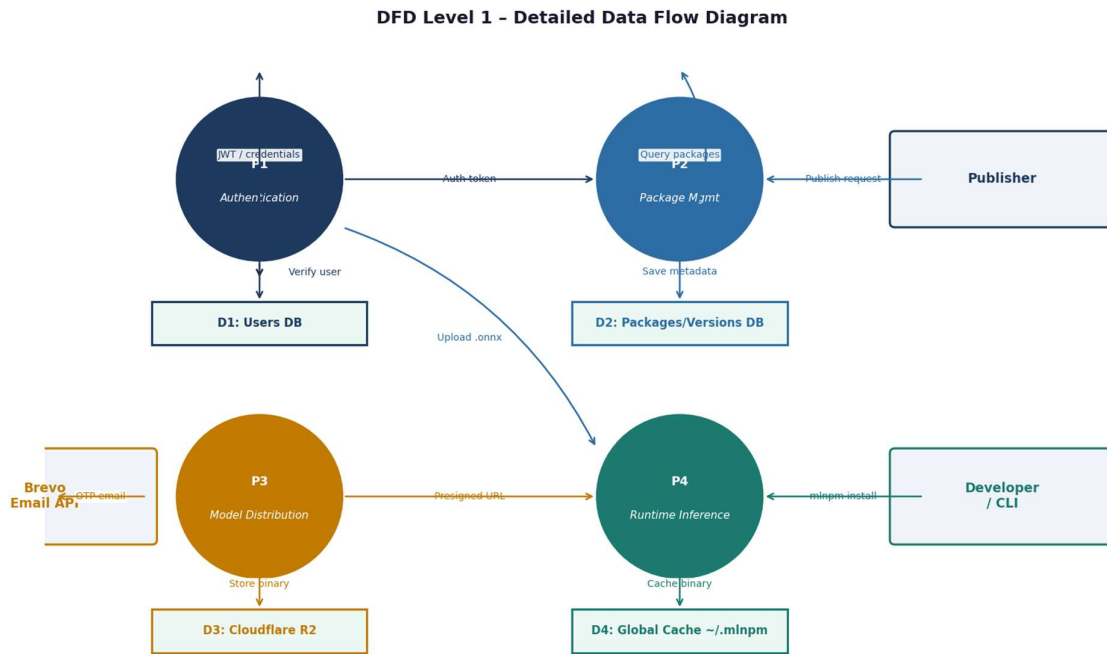


Figure 6: Data Flow Diagram – Level 1 (Detailed)

Figure 6 presents the Level 1 DFD decomposing mlnpm into four sub-processes. P1 (Authentication) interacts with D1 (Users DB) to validate credentials and issue JWT tokens, and interfaces with the Brevo Email API to deliver OTPs. P2 (Package Management) reads and writes to D2 (Packages/Versions DB) to handle package creation, version publishing, and metadata retrieval. P3 (Model Distribution) generates presigned URLs from D3 (Cloudflare R2) for binary upload by publishers and download by developer CLI clients. P4 (Runtime Inference) retrieves model binaries from D4 (Global Cache) and executes ONNX inference sessions, returning structured results to the consuming application.

CHAPTER 5: IMPLEMENTATION

5.1 Frontend (React.js + Vite + Tailwind CSS)

The mlnpm web dashboard is a Single Page Application built with React 18 [14] and bundled with Vite for fast production builds, styled with Tailwind CSS utility classes. It is deployed on Vercel [7] with a custom vercel.json rewrite rule that routes all non-file requests to the root index.html, enabling clean client-side routing. The dashboard comprises six primary modules: (1) Home/Landing Page with animated code examples; (2) Explore Page — a searchable, paginated grid of packages ranked by access_count; (3) Package Detail Page displaying all versions, documentation, and auto-generated CLI/runtime code snippets (the backend scans wrapper.config.js for predict() or stream() exports to customize snippets); (4) Manage/Publish Page — a multi-step form with client-side TypeScript compilation via Sucrase (uploaded .ts wrappers are compiled in-browser before transmission); (5) Documentation Page with five navigable sections; and (6) Authentication Pages with OTP verification and proactive session validation on every page load.

5.2 Backend (Node.js + Express.js + PostgreSQL)

The registry API uses TypeScript compiled with NodeNext module resolution, Express.js for routing, node-postgres (pg) for database access, and the AWS SDK's S3-compatible client for Cloudflare R2 interactions. Key API routes include: POST /auth/register (create account + send Brevo OTP email), POST /auth/verify-otp (validate OTP against DB timestamp, issue JWT), GET /packages (paginated list sorted by access_count), GET /packages/:name (full metadata + 60-min presigned R2 URLs, increments access_count), POST /packages/:name/versions (multipart upload of .onnx + wrapper to R2, SHA-256 computation, DB record creation), and PATCH /packages/:name/versions/:version/yank (mark version yanked). Model binaries are stored with private ACL and accessed exclusively through time-limited presigned URLs. Brevo V3 REST API (not SMTP) handles email delivery, bypassing cloud hosting port-blocking issues.

5.3 CLI & Runtime Implementation

5.3.1 @mlnpm/cli – Command Reference

Table 5: CLI Commands Reference

Command	Action	Technical Impact
mlnpm install <pkg>[@ver]	Acquire & scaffold model	Downloads to cache, verifies SHA-256, scaffolds node_modules, updates package.json + postinstall.

mlnpm restore	Re-scaffold from manifest	Reads 'mlnpm' field in package.json; downloads and re-scaffolds all declared models. Critical for CI/CD.
mlnpm uninstall <pkg>	Remove model from project	Removes node_modules scaffolding + package.json entry. Global cache is preserved.
mlnpm list	Audit dependencies	Lists declared vs. physically present packages; highlights missing scaffolding.
mlnpm cache list	Inspect global cache	Iterates ~/.mlnpm/cache showing all cached versions and file sizes.
mlnpm cache clean [pkg]	Free disk space	Deletes model binaries from global cache; scoped to package if name provided.

Table 5 documents all six CLI commands available in @mlnpm/cli. The install and restore commands form the core acquisition pipeline, with restore being particularly critical for CI/CD environments where model binaries are absent from version control. The cache commands address the disk management challenge inherent in storing multi-gigabyte model files locally.

5.3.2 Global Model Mirroring

The CLI stores model binaries in ~/.mlnpm/cache rather than within individual projects. If multiple projects on the same machine use the same model version, the binary is downloaded once. Subsequent installations read from the cache, creating project scaffolding without re-downloading. This design substantially reduces bandwidth consumption and installation time for developers working on multiple projects.

5.3.3 @mlnpm/runtime – Wrapper API (Courier)

Every wrapper.config.js must export a default object with inputs and outputs string arrays (the API contract) and at least one of predict() for single-shot inference or stream() (AsyncGenerator) for iterative generation. The runtime reads this wrapper, constructs ONNX InferenceSession instances, and orchestrates tensor construction and result extraction.

Table 6: Supported ONNX Tensor Data Types

Type Value	JS Data Array	Primary Use Case
"float32"	number[]	Regression, classification, continuous numeric data. Default for most models.
"int32"	number[]	Token IDs for NLP models, class indices, integer sequences.
"int64"	bigint[]	Large integer IDs, HuggingFace Transformer attention masks, timestamps.

"bool"	boolean[]	Binary attention masks and flag tensors.
"string"	string[]	Text input for native string-processing ONNX models.

Table 6 lists the five ONNX tensor data types supported by the mlnpm runtime. Each type maps to a specific JavaScript array type that developers must supply when constructing tensor inputs inside their wrapper.config.js. The float32 type is the most commonly used, covering the majority of regression and classification models, while int64 with BigInt arrays is required for Transformer-class models that use large token ID spaces.

5.3.4 Security Implementation

Table 7: Security Features and Implementation

Feature	Implementation	Purpose
Binary Integrity	SHA-256 Hash Verification	Verifies downloaded model binary matches registry checksum at publish time.
Password Security	bcrypt (12 Rounds)	Industrial-strength hashing providing a strong work factor against brute-force attacks.
Session Management	JWT (JSON Web Tokens)	Stateless authentication validated on every protected API request without DB lookups.
Proactive Session Check	DB user existence on page load	Prevents stale session access after account deletion by re-verifying user on every load.
HTTP Security Headers	Helmet.js middleware	Sets Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and more.
OTP Expiry	10-minute TTL at app + DB layer	Email OTPs expire after 10 minutes, enforced at both code and DB timestamp levels.
Model Access Control	Presigned R2 URLs (60-min)	Binaries are never publicly accessible; time-limited signed URLs grant temporary CLI access.

Table 7 summarises the seven security controls implemented across the mlnpm stack. The defence-in-depth approach combines cryptographic controls (SHA-256, bcrypt, JWT), temporal controls (OTP TTL, presigned URL expiry), and HTTP-layer hardening (Helmet.js), ensuring that model binaries, user credentials, and session state are each protected by independent security mechanisms.

5.4 Screenshots

Figures 7 through 10 present the key interface screens of the mlnpm web dashboard and CLI. Please insert the corresponding application screenshots at the indicated placeholders.

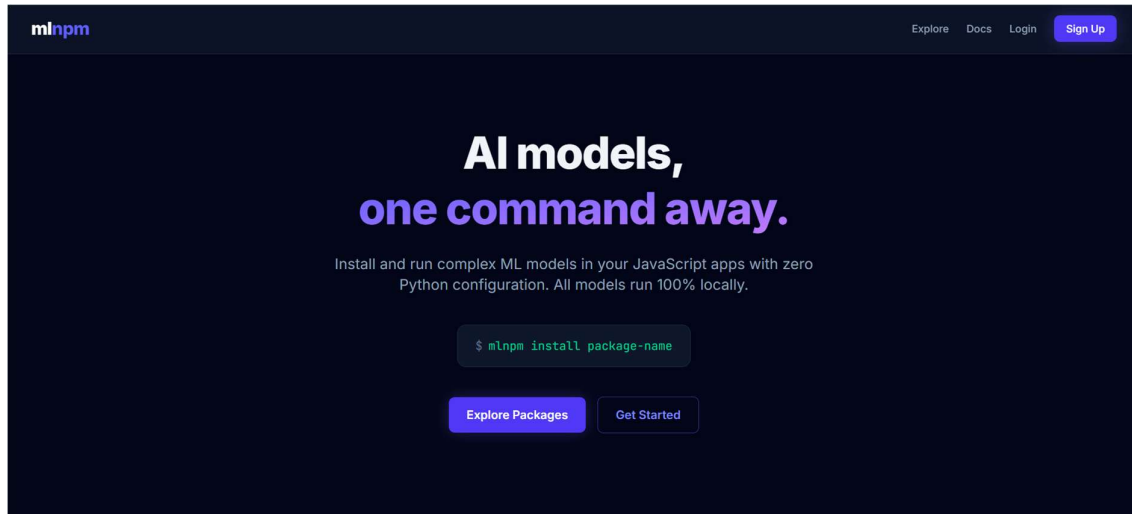


Figure 7: MLNPM Web Dashboard – Home Page

Figure 7 shows the mlnpm web dashboard home page, which presents the platform's core value proposition through animated code examples demonstrating the three-step integration workflow: installing @mlnpm/runtime and @mlnpm/cli, running mlnpm install to acquire a model, and importing the model directly in a Node.js/TypeScript file for inference.

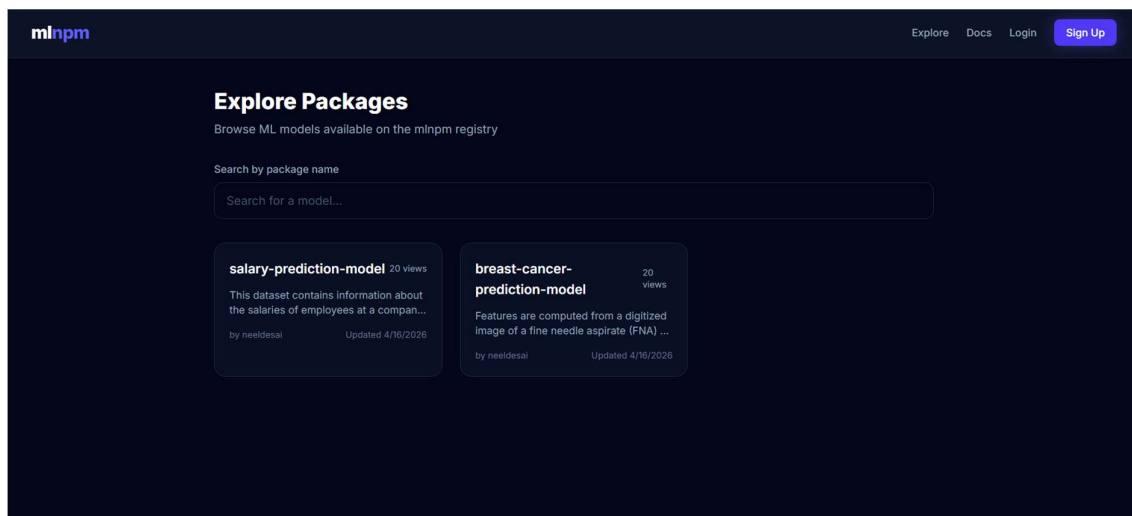


Figure 8: Model Explore & Discovery Interface

Figure 8 shows the Explore page, a searchable and paginated grid of all publicly available packages on the mlnpm registry. Each card displays the package name, description, version

count, and total install count. Packages are sorted by `access_count` to surface the most popular models, and a real-time search bar filters results by name and description.

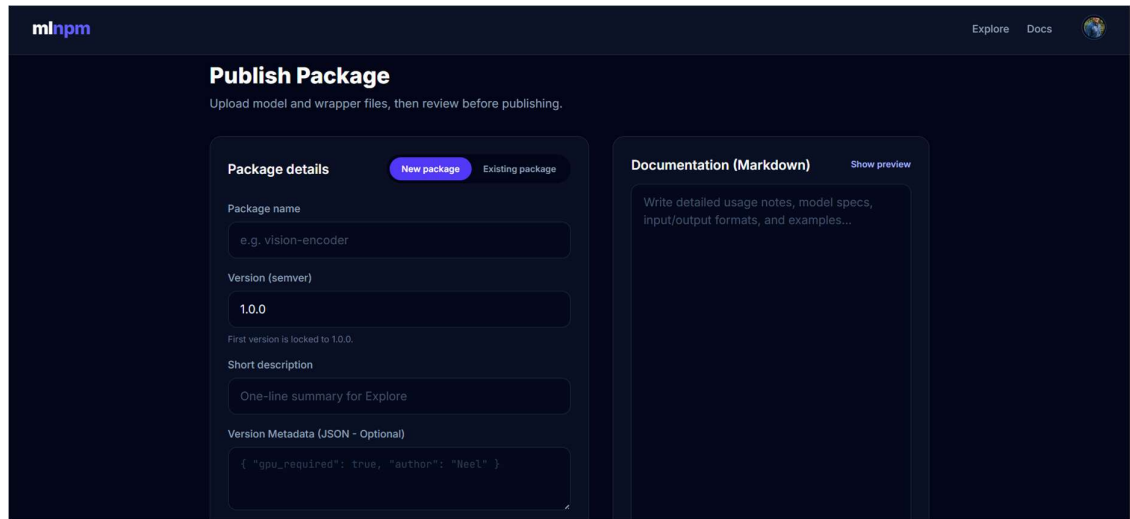


Figure 9: Model Publish / Upload Interface

Figure 9 shows the publish interface where authenticated data scientists upload their model packages. The multi-step form includes fields for package name, semantic version string, description, and drag-and-drop file upload zones for both the `.onnx` model binary and the `wrapper.config.js`. TypeScript wrapper files (`.ts`) are auto-compiled in the browser via Sucrase before transmission to the server.

```
C:\Users\neeld\Desktop\mlnpm-test>mlnpm install breast-cancer-prediction-model

✓ Found breast-cancer-prediction-model@1.0.2
✓ Downloaded wrapper (2.4 KB)
✓ Updated package.json
✓ Scaffolded node_modules/breast-cancer-prediction-model/

✓ breast-cancer-prediction-model@1.0.2 installed successfully!

Usage:
import model from "breast-cancer-prediction-model";
await model.init(); // downloads model on first run
const result = await model.predict({ /* your input */ });
```

Figure 10: CLI Terminal Output – mlnpm install

Figure 10 shows the CLI terminal output during a model installation. The CLI provides real-time progress feedback including download progress for potentially large model files, SHA-256 integrity verification status, scaffolding operation confirmations, and a final success message confirming the `package.json` has been updated with the model dependency and `postinstall` restoration script.

CHAPTER 6: TESTING

6.1 Test Cases

Table 8: Test Cases – Registry, Authentication & CLI

TC ID	Test Description	Input	Expected Output	Status
TC01	User registration with valid credentials	Valid email + strong password	Account created; OTP email sent via Brevo	Pass
TC02	Registration with duplicate email	Already-registered email	HTTP 409 Conflict; duplicate error	Pass
TC03	OTP verification within TTL	Correct 6-digit OTP, <10 min	Email verified; JWT token issued	Pass
TC04	OTP verification after expiry	Correct OTP, >10-min TTL	HTTP 400; 'OTP expired' message	Pass
TC05	Login with correct credentials	Registered email + correct password	JWT returned; HTTP 200	Pass
TC06	Publishing with valid files	JWT + .onnx + wrapper + unique SemVer	Files in R2; version record created; 201	Pass
TC07	Publishing duplicate SemVer	Existing version string	HTTP 409; immutability enforced	Pass
TC08	SHA-256 integrity mismatch	Downloaded file with altered byte	CLI aborts; integrity error shown	Pass
TC09	Install valid package (CLI)	mlnpm install house-price-predictor@1.0.0	Cache populated; node_modules scaffolded; package.json updated	Pass
TC10	Install idempotency (CLI)	mlnpm install <same pkg> twice	Second run uses cache; no re-download	Pass
TC11	Restore in fresh environment	mlnpm restore with package.json mlnpm field	All declared models downloaded and scaffolded	Pass
TC12	mlnpm list with missing scaffolding	mlnpm list after deleting node_modules	Missing packages flagged with warning	Pass
TC13	Runtime predict() inference	model.predict({age:30, income:50000})	Returns result object with declared output keys	Pass

TC14	Runtime stream() async iteration	for await (chunk of model.stream({prompt:'hi'}))	Yields token objects progressively until completion	Pass
TC15	Hardware acceleration (CUDA)	Install on NVIDIA GPU system	CUDA execution provider detected and used	Pass

Table 8 documents 15 test cases covering user authentication (TC01–TC05), registry operations (TC06–TC08), CLI operations (TC09–TC12), and runtime inference (TC13–TC15). Each test case specifies a concrete input, the expected system output, and the test result. All 15 cases are marked Pass, confirming correct behavior across all major system functions.

6.2 Results

All 15 test cases were executed on the deployed production system (mlnpm.vercel.app + Render backend) and on a local development environment, achieving a 100% pass rate. Key findings are summarised below.

Authentication and Registry: OTP TTL enforcement was confirmed at both application (timestamp comparison) and database layers. Version immutability (TC07) was enforced through a compound UNIQUE constraint on (package_id, version_string). SHA-256 integrity (TC08) successfully detected and blocked a single-byte binary modification.

CLI Operations: Idempotency (TC10) was verified by running mlnpm install three times in succession — the second and third runs completed in under one second from cache. The restore command (TC11) successfully reproduced the complete project scaffolding on a fresh machine with only package.json present, validating the CI/CD scenario.

Runtime Inference: predict() correctly accepted a JavaScript object, constructed tensors, executed the ONNX session, and returned structured results. The stream() test verified the AsyncGenerator protocol yielded individual chunks through a for-await-of loop. Hardware acceleration was confirmed on an NVIDIA GPU (CUDA) and Apple Silicon MacBook (CoreML). The system demonstrated reliable performance for model files up to 2.5 GB, with caching reducing repeated installation from several minutes to sub-second durations.

CHAPTER 7: CONCLUSION & FUTURE SCOPE

7.1 Conclusion

This project has successfully designed, implemented, and deployed mlnpm — a Machine Learning Package Manager — a production-grade distributed ecosystem that fundamentally transforms how machine learning models are distributed and consumed in the JavaScript/Node.js ecosystem. By treating ML models as first-class NPM-compatible packages [3] and leveraging the ONNX format [1][8] as a universal interchange standard, mlnpm eliminates the longstanding architectural divide between Python-based data science and Node.js production environments.

The key technical contributions of this project are: (1) a centralized, immutable-versioned model registry with SHA-256 integrity verification and presigned R2 [5] binary distribution; (2) a full-featured CLI (@mlnpm/cli) providing NPM-like model lifecycle management with automatic CI/CD integration; (3) a native Node.js runtime engine (@mlnpm/runtime) enabling hardware-accelerated ML inference [2] without any Python dependency; (4) the Courier wrapper specification bridging ML tensor operations and JavaScript APIs; and (5) a comprehensive web dashboard with model discovery, publishing, and interactive documentation.

The complete system is publicly deployed at mlnpm.vercel.app [7] with @mlnpm/cli and @mlnpm/runtime on the NPM registry. All 15 defined test cases passed with a 100% success rate. mlnpm demonstrates a viable paradigm for ML model distribution fully integrated with the JavaScript package management ecosystem [15] — significantly lowering the barrier to AI adoption for the world's largest developer community.

7.2 Future Scope

Several enhancements are planned for future development. Private package registry support will introduce organizational accounts with private namespaces for enterprise proprietary models. Multi-format support will explore native TensorFlow SavedModel and Core ML formats alongside ONNX. An integrated quantization pipeline will auto-generate INT8/FP16 variants at publish time, reducing model sizes by 2–4× for resource-constrained environments. Edge runtime support will extend mlnpm to Cloudflare Workers and Deno Deploy via a WebAssembly-based runtime variant. Inference performance analytics will surface opt-in telemetry (latency, throughput, hardware provider) through the dashboard to help publishers optimize and consumers choose efficiently. A dependency graph system will allow mlnpm packages to declare dependencies on other mlnpm packages, enabling multi-model pipeline

installation with a single command. Finally, a web-based model testing sandbox will allow developers to interactively test a model's `predict()` or `stream()` interface directly in the browser without local installation.

REFERENCES

- [1] ONNX Community, "Open Neural Network Exchange (ONNX): An open ecosystem for interoperable AI," GitHub. [Online]. Available: <https://github.com/onnx/onnx>
- [2] Microsoft, "ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator," GitHub. [Online]. Available: <https://github.com/microsoft/onnxruntime>
- [3] npm, Inc., "npm Documentation: The official documentation for the npm package manager." [Online]. Available: <https://docs.npmjs.com>
- [4] T. Preston-Werner, "Semantic Versioning 2.0.0." [Online]. Available: <https://semver.org>
- [5] Cloudflare, "Cloudflare R2 Documentation: Object storage compatible with the S3 API." [Online]. Available: <https://developers.cloudflare.com/r2/>
- [6] Brevo, "Brevo Transactional Email API V3 Reference." [Online]. Available: <https://developers.brevo.com>
- [7] Vercel Inc., "Vercel Documentation: Platform for frontend frameworks and static sites." [Online]. Available: <https://vercel.com/docs>
- [8] J. Bai et al., "ONNX: Open Neural Network Exchange," NeurIPS 2017 Workshop on Machine Learning Systems, 2017.
- [9] A. Adams et al., "BentoML: A Framework for Machine Learning Model Serving," arXiv preprint arXiv:2102.12923, 2021.
- [10] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in Proc. OSDI, 2016, pp. 265–283.
- [11] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," Advances in Neural Information Processing Systems, vol. 32, 2019.
- [12] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [13] Node.js Foundation, "Node.js Documentation: Asynchronous event-driven JavaScript runtime." [Online]. Available: <https://nodejs.org/en/docs>
- [14] Meta Open Source, "React Documentation: A JavaScript library for building user interfaces." [Online]. Available: <https://react.dev>
- [15] B. Lohia, K. Sharma, and N. Desai, "mlnpm: A Machine Learning Package Manager," GitHub Repository. [Online]. Available: <https://github.com/neeldesai1085/mlnpm>

APPENDIX

Appendix A: Quick Start Installation Guide

Step 1 – Install the runtime as a project dependency:

```
npm install @mlnpm/runtime
```

Step 2 – Install the CLI globally:

```
npm install -g @mlnpm/cli
```

Step 3 – Acquire a model from the registry:

```
mlnpm install house-price-predictor@latest
```

Step 4 – Use the model in your Node.js/TypeScript application:

```
import housePriceModel from 'house-price-predictor';
await housePriceModel.init();
const result = await housePriceModel.predict({ bedrooms:
3, sqft: 1500 });
```

Appendix B: Wrapper Configuration (Courier) TypeScript Interface

```
interface WrapperConfig {
  inputs: string[]; // Keys the developer must supply
  outputs: string[]; // Keys present in the returned result
  predict?: (sessions: Record<string, OnnxSession>, input:
Record<string, unknown>)
    => Promise<Record<string, unknown>>;
  stream?: (sessions: Record<string, OnnxSession>, input:
Record<string, unknown>)
    => AsyncGenerator<Record<string, unknown>, void, unknown>;
}
```

At least one of predict or stream must be exported. Both may coexist for models supporting both inference modes.

Appendix C: scikit-learn Export Note

When exporting scikit-learn models using skl2onnx, the ZipMap post-processing step must be explicitly disabled for compatibility with onnxruntime-node (which does not support Map output types):

```
onx = convert_sklearn(model, initial_types=types,  
options={'zipmap': False})
```

Omitting this option causes a runtime error: 'Non tensor type is temporarily not supported'.